StudyForge AI

Full Frontend Explanation

How the interface is built, connected to backend and made useful for students

React, Next.js, TypeScript, CSS, components, state and frontend viva

1 Frontend Purpose

The frontend is the visible student experience. It runs in the browser and turns backend data into buttons, forms, cards, chats and feedback. StudyForge frontend is more than a landing page: it handles account flow, PDF selection, streaming chat, library, planner, revision tracker, flashcards, quizzes, recall game, progress, theme and avatar customisation.

A good frontend has three jobs. It must make actions clear, keep track of changing interface state and communicate safely with the backend. It should not make important security decisions by itself. The backend remains the authority for accounts, documents and correct quiz answers.

User action	Frontend responsibility	Backend responsibility
Sign in	Collect form data and store returned token	Validate credentials and create token
Upload PDF	Build FormData and show notice	Validate, store and extract PDF
Ask chat	Render streaming text and sources	Retrieve chunks and call provider
Create quiz	Choose material and render options	Generate, validate and store questions
Save profile	Show form and success state	Persist profile and avatar

Viva question: Why is frontend important? Answer: It makes complex backend features understandable and usable for a student.

2 Frontend Technology Stack

Technology	What it does in StudyForge
Next.js 15	Project framework, build system and app folder structure.
React 19	Builds interface from reusable components and state.
TypeScript	Defines data shapes and catches mistakes early.
CSS	Controls warm visual style, responsive layout, dark mode and animation.
KaTeX and markdown packages	Render mathematics and formatted chat content.
Playwright	Runs browser-based UI checks.
Fetch API	Sends requests to FastAPI backend.

Next.js is built on React. React decides what should appear on screen from current state. CSS decides how it looks. TypeScript checks whether a Question really has ID, options and correct expected fields. The browser fetch function communicates with the API.

Viva question: What is the difference between Next.js and React? Answer: React is the UI library. Next.js is the larger framework that provides structure, development server and production build tools.

3 The App Folder and File Map

```
frontend/app/
  layout.tsx              Root HTML layout, styles, theme and motion
  page.tsx                Main client state, navigation and study tools
  ashiey-chat.tsx         Chat, streaming, messages, upload and saved chats
  library.tsx             Subject and PDF library
  planner.tsx             Task management UI
  revision.tsx            Revision topic UI
```

```
profile-setup.tsx          Profile and onboarding form
avatar-creator-3d.tsx      Gender-aware avatar creator
workspace-home.tsx         Dashboard and summary
workspace-header.tsx       Navigation header
study-api.ts               Shared calls to study router
message-content.tsx        Safe formatted response content
atelier.css etc.           Visual design and responsive styles
```

A component is separated when it has one clear purpose. page.tsx coordinates the whole workspace, but it does not contain every detail. This separation makes code easier to test and explain.

If examiner asks	Open this file
How login and page switch works	page.tsx
How chat works	ashiey-chat.tsx
How theme works	layout.tsx and theme-toggle.tsx
How avatar works	avatar-creator-3d.tsx and student-avatar-3d.tsx
How tasks work	planner.tsx
How library works	library.tsx
How formatting stays safe	message-content.tsx

4 React Components Props and State

A React component is a JavaScript or TypeScript function that returns interface elements. A component can receive props. Props are information passed from a parent component to a child. For example, Home gives token to Planner. Planner uses token to ask backend for tasks.

```
function Planner({ token }: { token: string }) {
  // use token for protected requests
  return <section>...</section>
}
```

State is data that can change while page is open. React useState returns current value and setter function. When setter runs, React renders again. This is why the page can switch from no quiz to a visible quiz after backend response.

React concept	Easy meaning	StudyForge example
Component	Reusable page part	Planner, Library, AshieyChat
Props	Data passed into component	token, user, onSaved
State	Changing data	quiz, busy, documentId
Setter	Safe state update function	setQuiz
Event handler	Runs after user action	onClick Create quiz
Render	React draws current state	Login screen or workspace

Viva question: What happens after setQuiz receives an array? Answer: React renders again and QuizView displays the generated questions.

5 page tsx Main Coordinator

page.tsx begins with use client because it uses browser features such as state, effects, local storage and event handlers. It imports specialised components. It declares API URL from NEXT_PUBLIC_API_URL, falling back to http://127.0.0.1:8000 for local development.

The Home component holds top-level state: token, user, auth mode, onboarding, active page, selected document, notice, cards, quiz and progress. This is called lifting state up. Components receive only the props they need.

```
const [token, setToken] = useState("")
const [user, setUser] = useState<User | null>(null)
const [page, setPage] = useState("hub")
const [documentId, setDocumentId] = useState("")
const [quiz, setQuiz] = useState<Question[]>([])
```

The component conditionally returns different screens. If there is no user it returns the login screen. If onboarding is true it returns profile setup. Otherwise it returns the full workspace shell with header, main content and footer.

Viva question: Why keep token at top level? Answer: Many child components need it for protected API calls, so the parent passes it down as a prop.

6 useEffect and Session Restore

useEffect runs after React renders. In StudyForge, one effect checks localStorage for studyforge_token on initial load. If a token exists, it calls GET /api/auth/me. If backend accepts it, user state is restored. If backend rejects it, frontend clears the invalid token.

A second effect watches token and page. When the user enters cards, quiz or games, it requests their document list. This means the material picker is ready when a study tool screen opens.

```
useEffect(() => {
  const saved = localStorage.getItem("studyforge_token")
  if (!saved) return
  fetch(API + "/api/auth/me", { headers: { Authorization: "Bearer " + saved } })
}, [])
```

The empty dependency array means run once after initial render. A dependency such as [token, page] means React runs effect again whenever either value changes.

Viva question: Why clear invalid token? Answer: Otherwise the page would keep sending a token the backend rejects and user would remain in a broken session.

7 Authentication Interface

The login and signup form is rendered in page.tsx when user is null. authMode decides whether name input appears. The auth function prevents default form reload, turns on busy state, calls signup or login route, reads JSON and handles error messages.

On success, frontend writes returned token to localStorage, sets token and user, and starts onboarding for new account. On failure, it places readable message in error state. This is a good example of handling success, loading and failure states.

State	Interface effect
busy true	Disable submit and show Please wait.
error text	Show error under form.
authMode signup	Show student name and Create workspace.
user null	Render auth screen.
onboarding true	Render profile setup before workspace.

Viva question: Why use event.preventDefault in auth form? Answer: It stops browser from reloading page so React can handle the request asynchronously.

8 Fetch API and Protected Requests

The browser uses fetch to call backend. A protected request includes Authorization Bearer token. A JSON request includes Content-Type application/json. A PDF upload uses FormData and intentionally does not manually set Content-Type because browser adds multipart boundary.

```
const response = await fetch(API + "/api/quizzes/generate", {
  method: "POST",
  headers: { "Content-Type": "application/json",
             Authorization: "Bearer " + token },
  body: JSON.stringify({ document_id: documentId, count: 5 })
})
const data = await response.json()
if (!response.ok) throw new Error(data.detail || "Could not generate.")
```

This pattern checks response.ok. A response can still contain JSON even when status is error. The code reads detail if server provides it and displays a student-friendly notice.

Viva question: Why not always assume fetch success? Answer: Network, authentication, validation and provider errors can all create non-success responses.

9 Upload and Material Picker

upload receives an optional File. It builds FormData and appends file under name file, matching backend UploadFile parameter. It sends POST /api/documents/upload with token. If upload succeeds, it saves document ID, document name and prepends the material to local list.

Cards, Quiz and Games share material picker in page.tsx. The selected document controls which PDF the backend uses. Changing material clears current cards and quiz, which avoids accidentally showing results from the previous document.

UI step	Why it exists
Choose PDF	Makes source material explicit.
Attach PDF	Allows upload from any study tool.
Busy disabled button	Avoids duplicate requests.
Notice text	Explains result or error.
Clear old set on change	Avoids mixed-document practice.

Viva question: Why does frontend store document ID instead of only filename? Answer: Backend needs unique document ID to find correct protected record.

10 Chat Interface and Streaming

ashiey-chat.tsx contains many state variables because chat has many moving parts: messages, input question, busy flag, reply status, documents, selected document, study mode, conversations, upload state, save status and scroll position.

The chat component requests chat status and documents, lets student select mode, then sends question to /api/chat/stream. It reads newline-delimited JSON events. Status updates a small message. Delta adds text to the current assistant message. Done saves final sources and conversation state. Error shows readable failure.

A ref points to transcript area so effect can scroll to latest message while a response streams. The code tracks whether user moved away from bottom so it does not force-scroll someone reading an earlier answer.

Event type	Frontend action

status	Show Reading your PDF or Thinking.
delta	Append text to assistant message.
done	Attach final answer, sources and save state.
error	Stop loading and show message.

Viva question: Why use a ref for transcript? Answer: A ref can access a DOM element such as scroll container without causing a new render.

11 Safe Message Content and Mathematics

message-content.tsx renders assistant content with react-markdown, remark-gfm, remark-math and rehype-katex. It supports headings, bullet lists, code blocks, tables and mathematical notation. A CodeBlock component adds a copy action and fallback status.

The important security point is that chat content is treated as content, not trusted raw HTML. The UI should not directly inject arbitrary HTML returned by a model. This reduces risk of script injection.

KaTeX is for rendering math such as formulas. It improves engineering study material where a student may ask about algorithms, probability or equations.

Viva question: Why not directly render raw model HTML? Answer: It could contain unsafe scripts or unwanted markup. Markdown rendering is safer and controlled.

12 Flashcards Quiz and Recall Views

CardsView receives cards and keeps local index plus revealed state. When cards change, an effect resets index and hide answer. It shows focus label, current card number, reveal button, navigation and shuffle action.

QuizView keeps answers object, result, error and busy state. Selecting an option updates answers by question ID. Submit sends answers to backend. Until result returns, student only sees question and options. After result, the UI shows server-returned answer, explanation and source page.

GameView uses the same card data as recall game. A tile hides or reveals answer. Reusing the card data avoids duplicate backend formats.

View	State it owns
CardsView	index and revealed
QuizView	answers, result, error and busy
GameView	which tile is revealed
Progress view	summary values from backend

Viva question: Why does QuizView keep answers as a dictionary? Answer: Each question ID maps to one selected option, making submission easy.

13 Library Planner Revision and Home

Library fetches subjects and documents from the study router. It supports adding subject, search, filter, assignment and selecting a document for chat. Planner fetches tasks, lets user create or edit a form, filters task list and can show calendar-oriented view. Revision fetches subjects and topics, then lets a student set learning, revision or confident state.

workspace-home.tsx requests summary and turns counts, tasks, documents and topic status into a quick dashboard. WorkspaceHeader provides navigation and closes dropdown details when user clicks elsewhere or presses Escape.

These components illustrate separation of concerns. They each own their interface state and call a small focused set of study API endpoints. page.tsx only passes token and callbacks.

Viva question: Why keep Planner in its own component? Answer: Task form, filters, calendar view and task API calls are complex enough to deserve a focused file.

14 Profile and 3D Styled Avatar

ProfileSetup begins with a form based on current user. It manages BTech branch, semester, college, subjects, goals and visual preferences. On save it calls profile API and uses onSaved callback so parent updates user state.

AvatarCreator uses initial avatar data plus default properties. It holds gender, optional self-description, busy, status and error. It also marks unsaved profile changes in document dataset and warns before browser unload. The parent navigation checks this state before leaving.

The 3D avatar is visual styling through React and CSS layers. It is not a WebGL or Three.js renderer. Gender choices control suggested hair choices but still allow student choice. This is a beginner-friendly implementation because it avoids a complicated graphics engine.

Viva question: Why does avatar creator track dirty state? Answer: It warns before accidental navigation loses unsaved customisation.

15 Theme Motion and Accessibility

layout.tsx imports CSS files and has a small script that reads saved color mode before normal app render. This reduces a flash of wrong theme. ThemeToggle changes a document data attribute and local storage. CSS uses the data attribute to apply dark or light colors.

MotionControl reads system reduced-motion preference. The project can limit decorative movement for students who find animation distracting or uncomfortable. Accessible design does not mean removing visual character; it means preserving control and understandable interactions.

Feature	Accessibility value
Theme toggle	Visual comfort in different environments.
Reduced motion	Avoid unnecessary movement.
Clear status text	Shows loading, error and completion.
Disabled busy buttons	Stops accidental repeat action.
Keyboard Escape in menu	Makes dropdown behavior easier.
Safe text rendering	Avoids unsafe raw content.

Viva question: What is reduced motion preference? Answer: A user or system setting that asks websites to reduce non-essential animation.

16 CSS Architecture and Responsive Layout

StudyForge uses several CSS files rather than putting every rule in one file. atelier.css creates warm visual identity. workspace.css structures workspace areas. dark.css handles theme changes. profile.css, study.css and studio.css focus on specific areas. This keeps styles understandable.

Responsive CSS lets interface adapt to phone and desktop width. A student may use laptop during study and phone for quick revision. CSS should allow cards and grids to wrap rather than force desktop widths.

Animation should support feedback. For example, hover, fade or gentle movement can show an interface is alive. It should not hide important information or create a barrier for reduced-motion users.

Viva question: Why separate CSS files? Answer: It groups styles by responsibility and prevents one giant hard-to-maintain stylesheet.

17 Frontend Errors Testing and Build

frontend/app/error.tsx gives a recovery screen if a page-level error happens. Individual components keep error state after failed requests. A good interface does not hide failures; it explains what action the student can take next.

npm.cmd run build creates a production Next.js build. It compiles TypeScript, checks framework structure and generates optimized output. Browser check scripts use Playwright and mock API calls so they can test user flows without creating a real account or calling Gemini.

Check	What it proves
Production build	Frontend compiles and types are valid.
Theme check	Theme behavior works.
Study check	Library, revision and chat related flows work.
Profile check	Profile setup flow works.
Workspace check	Major workspace navigation renders.

Viva question: Why is build useful before delivery? Answer: It catches compile and type errors that may not appear in a single development screen.

18 Frontend Viva Questions

Viva question: What is frontend? Answer: The browser-side interface used by the student.

Viva question: Why React? Answer: It builds reusable components and re-renders interface when state changes.

Viva question: What is component? Answer: A reusable UI function such as Planner or AshieyChat.

Viva question: What is prop? Answer: Data passed from parent component to child component.

Viva question: What is state? Answer: Changing data that controls what React renders.

Viva question: What is useEffect? Answer: A React hook for side effects such as fetching data or restoring session.

Viva question: Why TypeScript? Answer: It checks expected data shapes and prevents some coding mistakes.

Viva question: Why store token in local storage? Answer: To restore session in browser; backend still validates it on every protected request.

Viva question: Why send Bearer header? Answer: So backend can authenticate the user request.

Viva question: Why FormData for upload? Answer: It supports binary PDF file upload through multipart request.

Viva question: Why check response.ok? Answer: A fetch response may be JSON even when it is an error.

Viva question: Why stream chat? Answer: Student sees answer and status gradually rather than waiting.

Viva question: Why hide quiz answer initially? Answer: Backend preserves fairness until attempt is submitted.

Viva question: Is avatar WebGL? Answer: No, it is a 3D-styled React and CSS interface.

Viva question: What is future frontend improvement? Answer: More accessible keyboard flows, offline support, end-to-end tests and richer responsive analytics.

19 Frontend Walkthrough and Final Checklist

Study in this sequence. First open layout.tsx for global styling and theme. Next open page.tsx for state, auth, navigation, upload and generation. Then open AshieyChat for streaming. Read CardsView and QuizView inside page.tsx. After that open Library, Planner, Revision, ProfileSetup and AvatarCreator. Finally study CSS and Playwright checks.

- I can explain frontend versus backend.
- I can explain Next.js, React and TypeScript.
- I can explain component, prop, state and effect.
- I can explain page.tsx state and conditional rendering.
- I can explain login and token restore.
- I can explain Fetch API, JSON and FormData.
- I can explain chat streaming events.
- I can explain flashcard, quiz and recall views.
- I can explain library, planner and revision components.
- I can explain avatar, theme and reduced motion.
- I can explain build and browser testing.

Final Frontend Explanation

StudyForge frontend is a Next.js and React student workspace. It uses component-based design, TypeScript data types and state-driven rendering to give students a calm interface for PDFs, chat, cards, quizzes, planning and revision. It calls protected FastAPI routes through fetch, handles loading and error states, renders streamed chat safely, respects theme and motion preferences and keeps specialised features in focused components.